Swati Pillai

PRACTICAL FILE

TY BSc IT — Semester V (NEP 2020)

Subject: .NET Core and Progressive Web Application (Practical)

Software: Microsoft Visual Studio | OS: Windows

Module 1 & Module 2

Swati Pillai

MODULE 1

Swati Pillai

Practical 1a: MVC Application for Basic Arithmetic Operations

AIM

To create an ASP.NET Core MVC application that performs basic arithmetic operations (Addition, Subtraction, Multiplication, Division) using C#.

SOFTWARE REQUIRED

IDE	Microsoft Visual Studio 2022 / 2019
Framework	ASP.NET Core MVC (.NET 6 / .NET 8)
Language	C#
OS	Windows 10/11
Browser	Any modern browser (Chrome, Edge)

STEPS

Step 1: Create Project

1. Open Visual Studio → File → New → Project
2. Select 'ASP.NET Core Web App (Model-View-Controller)'
3. Name the project: ArithmeticMVC → Click Create
4. Select Framework: .NET 6.0 (or .NET 8.0) → Click Create

Step 2: Create the Model

Right-click Models folder → Add → Class → Name: ArithmeticModel.cs

PROGRAM CODE

Models/ArithmeticModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace ArithmeticMVC.Models
{
    public class ArithmeticModel
    {
        [Required]
        [Display(Name = "First Number")]
        public double Number1 { get; set; }

        [Required]
        [Display(Name = "Second Number")]
        public double Number2 { get; set; }

        public string Operation { get; set; } = "";
        public double Result { get; set; }
        public string ErrorMessage { get; set; } = "";
    }
}
```

Controllers/HomeController.cs

```
using ArithmeticMVC.Models;
using Microsoft.AspNetCore.Mvc;

namespace ArithmeticMVC.Controllers
```

Swati Pillai

```
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View(new ArithmeticModel());
        }

        [HttpPost]
        public IActionResult Index(ArithmeticModel model)
        {
            if (ModelState.IsValid)
            {
                switch (model.Operation)
                {
                    case "Add":
                        model.Result = model.Number1 + model.Number2;
                        break;
                    case "Subtract":
                        model.Result = model.Number1 - model.Number2;
                        break;
                    case "Multiply":
                        model.Result = model.Number1 * model.Number2;
                        break;
                    case "Divide":
                        if (model.Number2 == 0)
                        {
                            model.ErrorMessage = "Error: Division by zero!";
                        }
                        else
                        {
                            model.Result = model.Number1 / model.Number2;
                        }
                        break;
                }
            }
            return View(model);
        }
    }
}
```

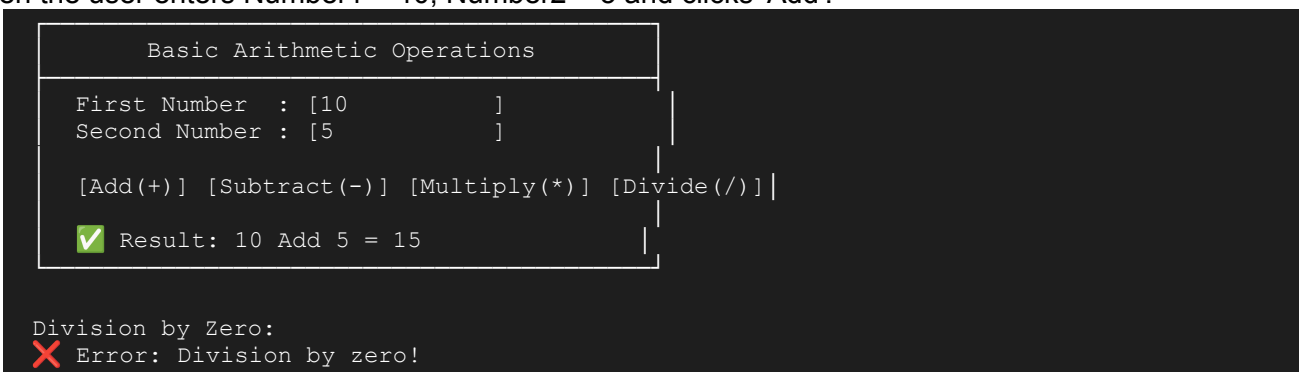
Views/Home/Index.cshtml

```
@model ArithmeticMVC.Models.ArithmeticModel
@{
    ViewData["Title"] = "Arithmetic Operations";
}

<h2 class="text-center mt-3">Basic Arithmetic Operations</h2>
<hr />

<div class="container">
  <form asp-action="Index" method="post">
    <div class="row justify-content-center">
      <div class="col-md-6">
        <div class="mb-3">
          <label asp-for="Number1" class="form-label"></label>
          <input asp-for="Number1" class="form-control" />
          <span asp-validation-for="Number1" class="text-danger"></span>
        </div>
        <div class="mb-3">
          <label asp-for="Number2" class="form-label"></label>
          <input asp-for="Number2" class="form-control" />
          <span asp-validation-for="Number2" class="text-danger"></span>
        </div>
        <div class="mb-3">
```

```
          <button type="submit" name="Operation" value="Add" class="btn btn-primary">Add
(+)</button>
          <button type="submit" name="Operation" value="Subtract" class="btn
btn-warning">Subtract (-)</button>
          <button type="submit" name="Operation" value="Multiply" class="btn
btn-success">Multiply (*)</button>
          <button type="submit" name="Operation" value="Divide" class="btn
btn-danger">Divide (/)</button>
        </div>
        @if (Model.ErrorMessage != "")
        {
          <div class="alert alert-danger">@Model.ErrorMessage</div>
        }
        else if (Model.Operation != "")
        {
          <div class="alert alert-success">
            <strong>Result:</strong> @Model.Number1 @Model.Operation @Model.Number2 =
@Model.Result
          </div>
        }
      </div>
    </div>
  </form>
</div>
```

OUTPUT

When the user enters Number1 = 10, Number2 = 5 and clicks 'Add':



Swati Pillai

Practical 1b: Floyd's Triangle till N Rows in C#

AIM

To write a C# program to print Floyd's triangle up to N rows, where N is input by the user.

SOFTWARE REQUIRED

IDE	Microsoft Visual Studio 2022
Type	C# Console Application
Framework	.NET 6 / .NET 8
OS	Windows 10/11

STEPS

1. Open Visual Studio → File → New → Project
2. Select 'Console App' → Name: FloydTriangle → Click Create
3. Write the following code in Program.cs
4. Press Ctrl+F5 to run the application

PROGRAM CODE

Program.cs

```
using System;

namespace FloydTriangle
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Floyd's Triangle ===");
            Console.Write("Enter the number of rows: ");
            int n = int.Parse(Console.ReadLine());

            int number = 1;

            Console.WriteLine("\nFloyd's Triangle:");
            Console.WriteLine(new string('-', 30));

            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(number + "\t");
                    number++;
                }
                Console.WriteLine();
            }

            Console.WriteLine(new string('-', 30));
            Console.WriteLine($"Total numbers printed: {number - 1}");
            Console.ReadKey();
        }
    }
}
```

Swati Pillai

OUTPUT

When user enters N = 5:

```
=== Floyd's Triangle ===
Enter the number of rows: 5

Floyd's Triangle:
------------------------------
1
2       3
4       5       6
7       8       9       10
11      12      13      14      15
------------------------------
Total numbers printed: 15
```

Swati Pillai

Practical 1c: Arrange Names of Cities in Sorted Order

AIM

To write a C# program that accepts names of 10 cities from the user and displays them in sorted (alphabetical) order.

SOFTWARE REQUIRED

IDE	Microsoft Visual Studio 2022
Type	C# Console Application
Framework	.NET 6 / .NET 8
OS	Windows 10/11

STEPS

1. Open Visual Studio → File → New → Project
2. Select 'Console App' → Name: SortCities → Click Create
3. Write the following code in Program.cs and run with Ctrl+F5

PROGRAM CODE

```
using System;

namespace SortCities
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] cities = new string[10];

            Console.WriteLine("=== City Name Sorter ===");
            Console.WriteLine("Enter names of 10 cities:");
            Console.WriteLine(new string('-', 30));

            for (int i = 0; i < 10; i++)
            {
                Console.Write($"City {i + 1}: ");
                cities[i] = Console.ReadLine();
            }

            // Sort array using Array.Sort()
            Array.Sort(cities, StringComparer.OrdinalIgnoreCase);

            Console.WriteLine("\n--- Cities in Sorted Order ---");
            for (int i = 0; i < cities.Length; i++)
            {
                Console.WriteLine($"{i + 1}. {cities[i]}");
            }

            Console.WriteLine(new string('-', 30));
            Console.ReadKey();
        }
    }
}
```

Swati Pillai

OUTPUT

```
=== City Name Sorter ===
Enter names of 10 cities:
------------------------------
City 1: Mumbai
City 2: Delhi
City 3: Pune
City 4: Chennai
City 5: Bangalore
City 6: Hyderabad
City 7: Kolkata
City 8: Ahmedabad
City 9: Jaipur
City 10: Surat

--- Cities in Sorted Order ---
1. Ahmedabad
2. Bangalore
3. Chennai
4. Delhi
5. Hyderabad
6. Jaipur
7. Kolkata
8. Mumbai
9. Pune
10. Surat
------------------------------
```

Swati Pillai

Practical 2: Features of C#

2a: Function Overloading

AIM

To demonstrate function overloading in C# by creating multiple methods with the same name but different parameters.

PROGRAM CODE

```
using System;

namespace FunctionOverloading
{
    class Calculator
    {
        // Overload 1: Two integers
        public int Add(int a, int b)
        {
            Console.WriteLine($"Add(int, int) called");
            return a + b;
        }

        // Overload 2: Three integers
        public int Add(int a, int b, int c)
        {
            Console.WriteLine($"Add(int, int, int) called");
            return a + b + c;
        }

        // Overload 3: Two doubles
        public double Add(double a, double b)
        {
            Console.WriteLine($"Add(double, double) called");
            return a + b;
        }

        // Overload 4: String concatenation
        public string Add(string a, string b)
        {
            Console.WriteLine($"Add(string, string) called");
            return a + b;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Calculator calc = new Calculator();
            Console.WriteLine("=== Function Overloading Demo ===");
            Console.WriteLine(calc.Add(10, 20));         // 30
            Console.WriteLine(calc.Add(10, 20, 30));     // 60
            Console.WriteLine(calc.Add(1.5, 2.5));       // 4.0
            Console.WriteLine(calc.Add("Hello, ", "World!"));
            Console.ReadKey();
        }
    }
}
```

OUTPUT

Swati Pillai

```
=== Function Overloading Demo ===
Add(int, int) called
30
Add(int, int, int) called
60
Add(double, double) called
4
Add(string, string) called
Hello, World!
```

2b: Inheritance (All Types)

AIM

To demonstrate all types of inheritance in C#: Single, Multilevel, Hierarchical, and Multiple Inheritance (via Interfaces).

PROGRAM CODE

```
using System;

namespace InheritanceDemo
{
    // ===== SINGLE INHERITANCE =====
    class Animal
    {
        public void Eat() => Console.WriteLine("Animal is eating");
    }
    class Dog : Animal   // Dog inherits Animal
    {
        public void Bark() => Console.WriteLine("Dog is barking");
    }

    // ===== MULTILEVEL INHERITANCE =====
    class GrandParent
    {
        public void Method1() => Console.WriteLine("GrandParent method");
    }
    class Parent : GrandParent
    {
        public void Method2() => Console.WriteLine("Parent method");
    }
    class Child : Parent   // Child -> Parent -> GrandParent
    {
        public void Method3() => Console.WriteLine("Child method");
    }

    // ===== HIERARCHICAL INHERITANCE =====
    class Shape
    {
        public void Draw() => Console.WriteLine("Drawing a shape");
    }
    class Circle : Shape
    {
        public void Info() => Console.WriteLine("This is a Circle");
    }
    class Rectangle : Shape
    {
        public void Info() => Console.WriteLine("This is a Rectangle");
    }

    // ===== MULTIPLE INHERITANCE via INTERFACES =====
    interface IFlyable
```

```
    {
        void Fly();
    }
    interface ISwimmable
    {
        void Swim();
    }
    class Duck : IFlyable, ISwimmable
    {
        public void Fly() => Console.WriteLine("Duck can fly");
        public void Swim() => Console.WriteLine("Duck can swim");
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("--- Single Inheritance ---");
            Dog d = new Dog();
            d.Eat(); d.Bark();

            Console.WriteLine("--- Multilevel Inheritance ---");
            Child c = new Child();
            c.Method1(); c.Method2(); c.Method3();

            Console.WriteLine("--- Hierarchical Inheritance ---");
            Circle cir = new Circle(); cir.Draw(); cir.Info();
            Rectangle rec = new Rectangle(); rec.Draw(); rec.Info();

            Console.WriteLine("--- Multiple Inheritance (Interface) ---");
            Duck duck = new Duck();
            duck.Fly(); duck.Swim();

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
--- Single Inheritance ---
Animal is eating
Dog is barking
--- Multilevel Inheritance ---
GrandParent method
Parent method
Child method
--- Hierarchical Inheritance ---
Drawing a shape
This is a Circle
Drawing a shape
This is a Rectangle
--- Multiple Inheritance (Interface) ---
Duck can fly
Duck can swim
```

2c: Constructor Overloading

AIM

To demonstrate constructor overloading in C# by defining multiple constructors with different parameter lists in the same class.

Swati Pillai

PROGRAM CODE

```
using System;

namespace ConstructorOverloading
{
    class Student
    {
        public string Name;
        public int Age;
        public string Course;

        // Default constructor
        public Student()
        {
            Name = "Unknown";
            Age = 0;
            Course = "N/A";
            Console.WriteLine("Default constructor called");
        }

        // Parameterized constructor with 1 argument
        public Student(string name)
        {
            Name = name;
            Age = 18;
            Course = "BSc IT";
            Console.WriteLine("1-param constructor called");
        }

        // Parameterized constructor with 2 arguments
        public Student(string name, int age)
        {
            Name = name;
            Age = age;
            Course = "BSc IT";
            Console.WriteLine("2-param constructor called");
        }

        // Parameterized constructor with all arguments
        public Student(string name, int age, string course)
        {
            Name = name;
            Age = age;
            Course = course;
            Console.WriteLine("3-param constructor called");
        }

        public void Display()
        {
            Console.WriteLine($"Name: {Name}, Age: {Age}, Course: {Course}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Constructor Overloading Demo ===");
            Student s1 = new Student();
            s1.Display();

            Student s2 = new Student("Alice");
            s2.Display();
```

```
            Student s3 = new Student("Bob", 20);
            s3.Display();

            Student s4 = new Student("Carol", 21, "BCA");
            s4.Display();

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Constructor Overloading Demo ===
Default constructor called
Name: Unknown, Age: 0, Course: N/A
1-param constructor called
Name: Alice, Age: 18, Course: BSc IT
2-param constructor called
Name: Bob, Age: 20, Course: BSc IT
3-param constructor called
Name: Carol, Age: 21, Course: BCA
```

2d: Interfaces

AIM

To demonstrate the use of interfaces in C# — defining contracts that classes must implement.

PROGRAM CODE

```
using System;

namespace InterfaceDemo
{
    // Define interfaces
    interface IShape
    {
        double Area();
        double Perimeter();
        void Display();
    }

    interface IPrintable
    {
        void Print();
    }

    class Circle : IShape, IPrintable
    {
        private double radius;
        public Circle(double r) { radius = r; }

        public double Area() => Math.PI * radius * radius;
        public double Perimeter() => 2 * Math.PI * radius;
        public void Display()
        {
            Console.WriteLine($"Circle: Radius={radius}");
            Console.WriteLine($"  Area      = {Area():F2}");
            Console.WriteLine($"  Perimeter = {Perimeter():F2}");
        }
        public void Print() => Console.WriteLine("[Printable] Circle Info Printed.");
    }
```

```
    class Rectangle : IShape
    {
        private double length, width;
        public Rectangle(double l, double w) { length = l; width = w; }

        public double Area() => length * width;
        public double Perimeter() => 2 * (length + width);
        public void Display()
        {
            Console.WriteLine($"Rectangle: Length={length}, Width={width}");
            Console.WriteLine($"  Area      = {Area():F2}");
            Console.WriteLine($"  Perimeter = {Perimeter():F2}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Interface Demo ===");
            IShape c = new Circle(7.0);
            c.Display();
            ((IPrintable)c).Print();

            Console.WriteLine();
            IShape r = new Rectangle(5.0, 3.0);
            r.Display();
            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Interface Demo ===
Circle: Radius=7
  Area      = 153.94
  Perimeter = 43.98
[Printable] Circle Info Printed.

Rectangle: Length=5, Width=3
  Area      = 15.00
  Perimeter = 16.00
```

2e: Using Delegates and Events

AIM

To demonstrate the use of delegates and events in C# for achieving callback mechanisms and event-driven programming.

PROGRAM CODE

```
using System;

namespace DelegatesEvents
{
    // Declare a delegate
    public delegate void NotifyHandler(string message);

    class EventPublisher
    {
        // Declare an event using the delegate
        public event NotifyHandler OnDataProcessed;
```

```
        public void ProcessData(string data)
        {
            Console.WriteLine($"Processing data: {data}");
            // Raise the event
            OnDataProcessed?.Invoke($"Data '{data}' has been processed!");
        }
    }

    class EventSubscriber
    {
        public void HandleEvent(string message)
        {
            Console.WriteLine($"[Subscriber] Received: {message}");
        }
    }

    // Simple delegate demo
    delegate int MathOperation(int x, int y);

    class Program
    {
        static int Multiply(int a, int b) => a * b;
        static int Add(int a, int b) => a + b;

        static void Main(string[] args)
        {
            Console.WriteLine("=== Delegates Demo ===");
            MathOperation op = Multiply;
            Console.WriteLine($"Multiply(4,5) = {op(4, 5)}");

            op = Add;
            Console.WriteLine($"Add(4,5) = {op(4, 5)}");

            Console.WriteLine("\n=== Events Demo ===");
            EventPublisher publisher = new EventPublisher();
            EventSubscriber subscriber = new EventSubscriber();

            // Subscribe to the event
            publisher.OnDataProcessed += subscriber.HandleEvent;
            publisher.OnDataProcessed += (msg) => Console.WriteLine($"[Lambda] {msg}");

            publisher.ProcessData("Student Record");
            publisher.ProcessData("Salary Data");

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Delegates Demo ===
Multiply(4,5) = 20
Add(4,5) = 9

=== Events Demo ===
Processing data: Student Record
[Subscriber] Received: Data 'Student Record' has been processed!
[Lambda] Data 'Student Record' has been processed!
Processing data: Salary Data
[Subscriber] Received: Data 'Salary Data' has been processed!
[Lambda] Data 'Salary Data' has been processed!
```

Swati Pillai

2f: Exception Handling

AIM

To demonstrate exception handling in C# using try-catch-finally blocks, custom exceptions, and multiple catch handlers.

PROGRAM CODE

```
using System;

namespace ExceptionHandling
{
    // Custom exception
    class InvalidAgeException : Exception
    {
        public InvalidAgeException(string msg) : base(msg) { }
    }

    class Program
    {
        static void ValidateAge(int age)
        {
            if (age < 0 || age > 120)
                throw new InvalidAgeException($"Invalid age: {age}");
        }

        static void Main(string[] args)
        {
            // 1. DivideByZeroException
            Console.WriteLine("--- DivideByZero Exception ---");
            try
            {
                int a = 10, b = 0;
                Console.WriteLine(a / b);
            }
            catch (DivideByZeroException ex)
            {
                Console.WriteLine($"Caught: {ex.Message}");
            }
            finally
            {
                Console.WriteLine("Finally block executed.");
            }

            // 2. FormatException
            Console.WriteLine("\n--- Format Exception ---");
            try
            {
                int num = int.Parse("abc");
            }
            catch (FormatException ex)
            {
                Console.WriteLine($"Caught: {ex.Message}");
            }

            // 3. IndexOutOfRange
            Console.WriteLine("\n--- IndexOutOfRange Exception ---");
            try
            {
                int[] arr = { 1, 2, 3 };
                Console.WriteLine(arr[10]);
            }
            catch (IndexOutOfRangeException ex)
```

```
            {
                Console.WriteLine($"Caught: {ex.Message}");
            }

            // 4. Custom Exception
            Console.WriteLine("\n--- Custom Exception ---");
            try
            {
                ValidateAge(-5);
            }
            catch (InvalidAgeException ex)
            {
                Console.WriteLine($"Custom Exception: {ex.Message}");
            }

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
--- DivideByZero Exception ---
Caught: Attempted to divide by zero.
Finally block executed.

--- Format Exception ---
Caught: Input string was not in a correct format.

--- IndexOutOfRange Exception ---
Caught: Index was outside the bounds of the array.

--- Custom Exception ---
Custom Exception: Invalid age: -5
```

Swati Pillai

Practical 3: Delegates, Multicast Delegates, and Properties

3a: Arithmetic Methods via C# Delegates

AIM

To create arithmetic methods add(), multiply(), subtract(), divide() and call them using C# delegates.

PROGRAM CODE

```
using System;

namespace DelegateArithmetic
{
    delegate double MathOp(double a, double b);

    class Arithmetic
    {
        public static double Add(double a, double b)      => a + b;
        public static double Multiply(double a, double b) => a * b;
        public static double Subtract(double a, double b) => a - b;
        public static double Divide(double a, double b)
        {
            if (b == 0) throw new DivideByZeroException();
            return a / b;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Arithmetic via Delegates ===");
            double x = 10, y = 5;

            MathOp op = Arithmetic.Add;
            Console.WriteLine($"Add({x},{y})      = {op(x, y)}");

            op = Arithmetic.Subtract;
            Console.WriteLine($"Subtract({x},{y}) = {op(x, y)}");

            op = Arithmetic.Multiply;
            Console.WriteLine($"Multiply({x},{y}) = {op(x, y)}");

            op = Arithmetic.Divide;
            Console.WriteLine($"Divide({x},{y})   = {op(x, y)}");

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Arithmetic via Delegates ===
Add(10,5)      = 15
Subtract(10,5) = 5
Multiply(10,5) = 50
Divide(10,5)   = 2
```

Swati Pillai

3b: Multicast Delegate

AIM

To demonstrate a multicast delegate in C# that holds references to multiple methods and invokes them all in sequence.

PROGRAM CODE

```
using System;

namespace MulticastDelegate
{
    delegate void PrintDelegate(string message);

    class Printer
    {
        public static void PrintUpperCase(string msg)
        {
            Console.WriteLine($"UPPER: {msg.ToUpper()}");
        }

        public static void PrintLowerCase(string msg)
        {
            Console.WriteLine($"lower: {msg.ToLower()}");
        }

        public static void PrintReverse(string msg)
        {
            char[] chars = msg.ToCharArray();
            Array.Reverse(chars);
            Console.WriteLine($"Reversed: {new string(chars)}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Multicast Delegate Demo ===");

            PrintDelegate del = Printer.PrintUpperCase;
            del += Printer.PrintLowerCase;
            del += Printer.PrintReverse;

            Console.WriteLine("Invoking multicast delegate:");
            del("Hello World");

            Console.WriteLine("\nRemoving PrintLowerCase:");
            del -= Printer.PrintLowerCase;
            del("Hello World");

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Multicast Delegate Demo ===
Invoking multicast delegate:
UPPER: HELLO WORLD
lower: hello world
Reversed: dlroW olleH
```

```
Removing PrintLowerCase:
UPPER: HELLO WORLD
Reversed: dlroW olleH
```

3c: BankAccount Class with Property Validation

AIM

To create a BankAccount class with AccountNumber and Balance properties, where the Balance property includes validation to prevent negative balances.

PROGRAM CODE

```
using System;

namespace BankAccountApp
{
    class BankAccount
    {
        private string accountNumber;
        private double balance;

        public string AccountNumber
        {
            get { return accountNumber; }
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Account number cannot be empty!");
                accountNumber = value;
            }
        }

        public double Balance
        {
            get { return balance; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Balance cannot be negative!");
                balance = value;
            }
        }

        public BankAccount(string accNo, double initialBalance)
        {
            AccountNumber = accNo;
            Balance = initialBalance;
        }

        public void Deposit(double amount)
        {
            if (amount <= 0) throw new ArgumentException("Deposit must be positive.");
            Balance += amount;
            Console.WriteLine($"Deposited Rs.{amount}. New Balance: Rs.{Balance}");
        }

        public void Withdraw(double amount)
        {
            if (amount > Balance) throw new InvalidOperationException("Insufficient
funds!");
            Balance -= amount;
```

```
            Console.WriteLine($"Withdrawn Rs.{amount}. New Balance: Rs.{Balance}");
        }

        public void DisplayInfo()
        {
            Console.WriteLine($"Account: {AccountNumber} | Balance: Rs.{Balance}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Bank Account Demo ===");
            BankAccount acc = new BankAccount("ACC1001", 5000);
            acc.DisplayInfo();
            acc.Deposit(2000);
            acc.Withdraw(1500);
            acc.DisplayInfo();

            Console.WriteLine("\nTrying negative balance:");
            try { acc.Balance = -100; }
            catch (Exception ex) { Console.WriteLine($"Error: {ex.Message}"); }

            Console.WriteLine("\nTrying overdraft:");
            try { acc.Withdraw(50000); }
            catch (Exception ex) { Console.WriteLine($"Error: {ex.Message}"); }

            Console.ReadKey();
        }
    }
}
```

OUTPUT

```
=== Bank Account Demo ===
Account: ACC1001 | Balance: Rs.5000
Deposited Rs.2000. New Balance: Rs.7000
Withdrawn Rs.1500. New Balance: Rs.5500
Account: ACC1001 | Balance: Rs.5500

Trying negative balance:
Error: Balance cannot be negative!

Trying overdraft:
Error: Insufficient funds!
```

Swati Pillai

Practical 4: ASP.NET Web Forms Application

4a: Student Registration Form

AIM

To design a Student Registration Form using ASP.NET Web Forms controls including Label, TextBox, RadioButton, DropDownList, CheckBox, Calendar, and Button.

STEPS

1. Open Visual Studio → File → New → Project → ASP.NET Web Application (.NET Framework)
2. Select 'Web Forms' template → Name: StudentRegForm → Click Create
3. Open Default.aspx and add the following markup
4. Open Default.aspx.cs and add the code-behind logic
5. Press F5 to run the application

PROGRAM CODE

Default.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
   CodeBehind="Default.aspx.cs" Inherits="StudentRegForm.Default" %>

<!DOCTYPE html>
<html>
<head runat="server">
    <title>Student Registration</title>
    <style>
        body { font-family: Arial; margin: 30px; }
        .form-group { margin: 10px 0; }
        label { display: inline-block; width: 180px; font-weight: bold; }
    </style>
</head>
<body>
  <form id="form1" runat="server">
    <h2>Student Registration Form</h2>
    <hr />
    <div class="form-group">
      <asp:Label Text="Student Name:" runat="server" />
      <asp:TextBox ID="txtName" runat="server" Width="200px" />
    </div>
    <div class="form-group">
      <asp:Label Text="Gender:" runat="server" />
      <asp:RadioButton ID="rbMale" GroupName="gender" Text="Male" runat="server" />
      <asp:RadioButton ID="rbFemale" GroupName="gender" Text="Female" runat="server" />
    </div>
    <div class="form-group">
      <asp:Label Text="Course:" runat="server" />
      <asp:DropDownList ID="ddlCourse" runat="server">
        <asp:ListItem Text="Select Course" Value="" />
        <asp:ListItem Text="BSc IT" Value="BSc IT" />
        <asp:ListItem Text="BSc CS" Value="BSc CS" />
        <asp:ListItem Text="BCA" Value="BCA" />
      </asp:DropDownList>
    </div>
    <div class="form-group">
      <asp:Label Text="Date of Birth:" runat="server" />
      <asp:Calendar ID="calDOB" runat="server" />
    </div>
    <div class="form-group">
```

```
      <asp:CheckBox ID="chkTerms" Text="I accept Terms and Conditions" runat="server" />
    </div>
    <div class="form-group">
      <asp:Button ID="btnSubmit" Text="Register" runat="server"
           OnClick="btnSubmit_Click" />
      <asp:Button ID="btnClear" Text="Clear" runat="server"
           OnClick="btnClear_Click" />
    </div>
    <hr />
    <asp:Label ID="lblOutput" runat="server" ForeColor="Green" Font-Bold="true" />
  </form>
</body>
</html>
```

Default.aspx.cs

```
using System;

namespace StudentRegForm
{
    public partial class Default : System.Web.UI.Page
    {
        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            if (!chkTerms.Checked)
            {
                lblOutput.ForeColor = System.Drawing.Color.Red;
                lblOutput.Text = "Please accept Terms and Conditions!";
                return;
            }

            string gender = rbMale.Checked ? "Male" : (rbFemale.Checked ? "Female" : "Not
Selected");
            string dob = calDOB.SelectedDate == DateTime.MinValue
                ? "Not Selected"
                : calDOB.SelectedDate.ToShortDateString();

            lblOutput.ForeColor = System.Drawing.Color.Green;
            lblOutput.Text = $"Registration Successful!<br/>"
                           + $"Name: {txtName.Text}<br/>"
                           + $"Gender: {gender}<br/>"
                           + $"Course: {ddlCourse.SelectedValue}<br/>"
                           + $"DOB: {dob}";
        }

        protected void btnClear_Click(object sender, EventArgs e)
        {
            txtName.Text = "";
            rbMale.Checked = false;
            rbFemale.Checked = false;
            ddlCourse.SelectedIndex = 0;
            chkTerms.Checked = false;
            lblOutput.Text = "";
        }
    }
}
```

OUTPUT

```
╔═══════════════════════════════════════╗
║      Student Registration Form        ║
╠═══════════════════════════════════════╣
║ Student Name:  [Alice Sharma        ] ║
║ Gender:        (•) Male  ( ) Female   ║
║ Course:        [BSc IT ▼]             ║
║ Date of Birth: [Calendar Widget]      ║
║ [✓] I accept Terms and Conditions     ║
║ [Register]  [Clear]                   ║
```

```
Registration Successful!
Name: Alice Sharma
Gender: Male
Course: BSc IT
DOB: 06/10/2003
```

4b: Registration Form with Validation Controls

AIM

To create a Registration Form using ASP.NET validation controls: RequiredFieldValidator, RangeValidator, RegularExpressionValidator, and CompareValidator.

PROGRAM CODE

ValidationForm.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Validation.aspx.cs"
   Inherits="StudentRegForm.Validation" %>
<html>
<head runat="server"><title>Validation Demo</title></head>
<body>
  <form id="form1" runat="server">
    <h2>Registration with Validation</h2>
    <table>
      <tr>
        <td>Name:</td>
        <td><asp:TextBox ID="txtName" runat="server" /></td>
        <td>
          <asp:RequiredFieldValidator ControlToValidate="txtName"
            ErrorMessage="Name is required!" ForeColor="Red" runat="server" />
        </td>
      </tr>
      <tr>
        <td>Age:</td>
        <td><asp:TextBox ID="txtAge" runat="server" /></td>
        <td>
          <asp:RequiredFieldValidator ControlToValidate="txtAge"
            ErrorMessage="Age required!" ForeColor="Red" runat="server" />
          <asp:RangeValidator ControlToValidate="txtAge" Type="Integer"
            MinimumValue="18" MaximumValue="60"
            ErrorMessage="Age must be 18-60" ForeColor="Red" runat="server" />
        </td>
      </tr>
      <tr>
        <td>Email:</td>
        <td><asp:TextBox ID="txtEmail" runat="server" /></td>
        <td>
          <asp:RegularExpressionValidator ControlToValidate="txtEmail"
            ValidationExpression="^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$"
            ErrorMessage="Invalid Email!" ForeColor="Red" runat="server" />
        </td>
      </tr>
      <tr>
        <td>Password:</td>
        <td><asp:TextBox ID="txtPass" TextMode="Password" runat="server" /></td>
        <td>
          <asp:RequiredFieldValidator ControlToValidate="txtPass"
            ErrorMessage="Password required!" ForeColor="Red" runat="server" />
        </td>
      </tr>
      <tr>
        <td>Confirm:</td>
```

```
        <td><asp:TextBox ID="txtConfirm" TextMode="Password" runat="server" /></td>
        <td>
          <asp:CompareValidator ControlToValidate="txtConfirm"
            ControlToCompare="txtPass"
            ErrorMessage="Passwords do not match!" ForeColor="Red" runat="server" />
        </td>
      </tr>
      <tr>
        <td colspan="3">
          <asp:Button ID="btnReg" Text="Register" runat="server"
                OnClick="btnReg_Click" />
        </td>
      </tr>
    </table>
    <asp:ValidationSummary runat="server" ForeColor="Red" />
    <asp:Label ID="lblMsg" runat="server" ForeColor="Green" />
  </form>
</body>
</html>
```

Validation.aspx.cs

```
protected void btnReg_Click(object sender, EventArgs e)
{
    if (Page.IsValid)
    {
        lblMsg.Text = $"Welcome {txtName.Text}! Registration successful.";
    }
}
```

OUTPUT

```
When submitted with empty fields:
• Name is required!
• Age required!
• Password required!

When age = 15:
• Age must be 18-60

When passwords don't match:
• Passwords do not match!

On success:
Welcome John! Registration successful.
```

4c: AdRotator Control

AIM

To create a Web Form that demonstrates the use of ASP.NET AdRotator control to display rotating advertisements from an XML file.

PROGRAM CODE

Ads.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Advertisements>
  <Ad>
    <ImageUrl>~/images/ad1.jpg</ImageUrl>
    <NavigateUrl>https://www.google.com</NavigateUrl>
    <AlternateText>Google Ad</AlternateText>
    <Impressions>50</Impressions>
  </Ad>
  <Ad>
    <ImageUrl>~/images/ad2.jpg</ImageUrl>
```

```
    <NavigateUrl>https://www.microsoft.com</NavigateUrl>
    <AlternateText>Microsoft Ad</AlternateText>
    <Impressions>30</Impressions>
  </Ad>
  <Ad>
    <ImageUrl>~/images/ad3.jpg</ImageUrl>
    <NavigateUrl>https://www.amazon.com</NavigateUrl>
    <AlternateText>Amazon Ad</AlternateText>
    <Impressions>20</Impressions>
  </Ad>
</Advertisements>
```

AdRotatorDemo.aspx

```
<html>
<head runat="server"><title>AdRotator Demo</title></head>
<body>
  <form id="form1" runat="server">
    <h2>AdRotator Control Demo</h2>
    <p>Refresh the page to see a different advertisement:</p>
    <asp:AdRotator ID="AdRotator1" runat="server"
        AdvertisementFile="~/Ads.xml"
        Width="468px" Height="60px" />
    <br /><br />
    <asp:Button ID="btnRefresh" Text="Show Another Ad" runat="server" />
  </form>
</body>
</html>
```

OUTPUT

```
AdRotator Control Demo
Refresh the page to see a different advertisement:

 ____________________________________________
|    [Google Advertisement Banner Image]     |
|____________________________________________|

(On next refresh / button click:)
 ____________________________________________
|    [Microsoft Advertisement Banner Image]  |
|____________________________________________|
```

Swati Pillai

Practical 5: Cookies, EF Core, CRUD, and Authentication

5a: Different Types of Cookies

AIM

To create an ASP.NET Core web application demonstrating Session Cookies and Persistent Cookies.

PROGRAM CODE

Controllers/CookieController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace CookieDemo.Controllers
{
    public class CookieController : Controller
    {
        // Set Session Cookie (expires when browser closes)
        public IActionResult SetSessionCookie()
        {
            Response.Cookies.Append("SessionUser", "Alice");
            return Content("Session cookie 'SessionUser' set!");
        }

        // Set Persistent Cookie (expires after 7 days)
        public IActionResult SetPersistentCookie()
        {
            var options = new CookieOptions
            {
                Expires = DateTime.Now.AddDays(7),
                IsEssential = true,
                HttpOnly = true,
                Secure = true
            };
            Response.Cookies.Append("PersistentUser", "Bob", options);
            return Content("Persistent cookie 'PersistentUser' set for 7 days!");
        }

        // Read Cookies
        public IActionResult ReadCookies()
        {
            string session = Request.Cookies["SessionUser"] ?? "Not found";
            string persistent = Request.Cookies["PersistentUser"] ?? "Not found";
            return Content($"Session Cookie: {session}\nPersistent Cookie:
{persistent}");
        }

        // Delete Cookie
        public IActionResult DeleteCookie()
        {
            Response.Cookies.Delete("SessionUser");
            Response.Cookies.Delete("PersistentUser");
            return Content("Cookies deleted!");
        }
    }
}
```

OUTPUT

```
/Cookie/SetSessionCookie  → Session cookie 'SessionUser' set!
/Cookie/SetPersistentCookie → Persistent cookie 'PersistentUser' set for 7 days!
/Cookie/ReadCookies →
```

```
    Session Cookie: Alice
    Persistent Cookie: Bob
/Cookie/DeleteCookie  → Cookies deleted!
```

5b: Product Table with EF Core (Insert Operation)

AIM

To create a Product table in SQL Server and perform Insert Operation using Entity Framework Core in an ASP.NET Core application.

STEPS

1. Install NuGet packages: Microsoft.EntityFrameworkCore, Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.Tools
2. Create the Product model and DbContext
3. Configure connection string in appsettings.json
4. Run migrations: Add-Migration Init → Update-Database

PROGRAM CODE

Models/Product.cs

```
namespace EFCoreDemo.Models
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; }
        public string Category { get; set; }
        public decimal Price { get; set; }
        public int Stock { get; set; }
    }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using EFCoreDemo.Models;

namespace EFCoreDemo.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Product> Products { get; set; }
    }
}
```

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using EFCoreDemo.Data;
using EFCoreDemo.Models;

namespace EFCoreDemo.Controllers
{
    public class ProductController : Controller
    {
        private readonly AppDbContext _context;
        public ProductController(AppDbContext context) { _context = context; }

        public IActionResult Create()
        {
```

```
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Create(Product product)
        {
            if (ModelState.IsValid)
            {
                _context.Products.Add(product);
                await _context.SaveChangesAsync();
                TempData["Success"] = "Product added successfully!";
                return RedirectToAction("Index");
            }
            return View(product);
        }

        public async Task<IActionResult> Index()
        {
            var products = await _context.Products.ToListAsync();
            return View(products);
        }
    }
}
```

OUTPUT

```
                Product List
┌────┬──────────┬──────────┬───────┬───────┐
│ ID │ Name     │ Category │ Price │ Stock │
├────┼──────────┼──────────┼───────┼───────┤
│  1 │ Laptop   │ Elec.    │ 55000 │ 10    │
│  2 │ Mouse    │ Elec.    │   500 │ 50    │
└────┴──────────┴──────────┴───────┴───────┘
✅ Product added successfully!
```

5c: CRUD Operations on Employee Table using EF Core

AIM

To develop a web application that performs Create, Read, Update, and Delete (CRUD) operations on an Employee table using Entity Framework Core and ASP.NET Core MVC.

PROGRAM CODE

Models/Employee.cs

```
using System.ComponentModel.DataAnnotations;

namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int EmployeeId { get; set; }

        [Required] public string Name { get; set; }
        [Required] public string Department { get; set; }
        [Range(0, 1000000)] public decimal Salary { get; set; }
        public string Email { get; set; }
    }
}
```

Controllers/EmployeeController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
```

Swati Pillai

```
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Controllers
{
    public class EmployeeController : Controller
    {
        private readonly AppDbContext _ctx;
        public EmployeeController(AppDbContext ctx) { _ctx = ctx; }

        // READ (Index)
        public async Task<IActionResult> Index()
            => View(await _ctx.Employees.ToListAsync());

        // CREATE
        public IActionResult Create() => View();

        [HttpPost]
        public async Task<IActionResult> Create(Employee emp)
        {
            if (ModelState.IsValid)
            {
                _ctx.Employees.Add(emp);
                await _ctx.SaveChangesAsync();
                return RedirectToAction(nameof(Index));
            }
            return View(emp);
        }

        // UPDATE
        public async Task<IActionResult> Edit(int id)
        {
            var emp = await _ctx.Employees.FindAsync(id);
            if (emp == null) return NotFound();
            return View(emp);
        }

        [HttpPost]
        public async Task<IActionResult> Edit(Employee emp)
        {
            if (ModelState.IsValid)
            {
                _ctx.Employees.Update(emp);
                await _ctx.SaveChangesAsync();
                return RedirectToAction(nameof(Index));
            }
            return View(emp);
        }

        // DELETE
        public async Task<IActionResult> Delete(int id)
        {
            var emp = await _ctx.Employees.FindAsync(id);
            if (emp == null) return NotFound();
            return View(emp);
        }

        [HttpPost, ActionName("Delete")]
        public async Task<IActionResult> DeleteConfirmed(int id)
        {
            var emp = await _ctx.Employees.FindAsync(id);
            _ctx.Employees.Remove(emp);
            await _ctx.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
    }
```

```
}
```

OUTPUT

```
Employee List (Index Page)

| ID | Name          | Department | Salary | Actions  |
|----|---------------|------------|--------|----------|
|  1 | Rahul Sharma  | IT         | 45000  | Edit|Del |
|  2 | Priya Mehta   | HR         | 38000  | Edit|Del |

[+ Add New Employee]
✅ Record saved / updated / deleted successfully.
```

5d: Authentication and Authorization in ASP.NET Core

AIM

To implement user Authentication and Role-based Authorization in an ASP.NET Core MVC application using ASP.NET Core Identity.

STEPS

1. Create ASP.NET Core MVC project with 'Individual User Accounts' authentication
2. Install package: Microsoft.AspNetCore.Identity.EntityFrameworkCore
3. Run migrations: Update-Database
4. Add [Authorize] attribute to protected controllers

PROGRAM CODE

Program.cs (relevant parts)

```
builder.Services.AddDbContext<ApplicationDbContext>(...);
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 8;
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();

// After app.Build()
app.UseAuthentication();
app.UseAuthorization();
```

Controllers/AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
    public IActionResult Dashboard()
    {
        return View();
    }
}

[Authorize]   // Any logged-in user
public class ProfileController : Controller
{
    public IActionResult Index()
    {
```

```
            return View();
        }
    }
```

OUTPUT

```
Login Page:
╔═══════════════════════════════╗
║          Log In               ║
║   Email: [user@example.com]   ║
║   Password: [**********]      ║
║   [Log in]                    ║
╚═══════════════════════════════╝

Unauthorized access attempt:
→ Redirected to /Account/Login?returnUrl=%2FAdmin%2FDashboard

After login (as Admin):
→ Admin Dashboard accessible ✅
Non-admin user accessing /Admin/Dashboard:
→ 403 Forbidden ❌
```

Swati Pillai

MODULE 2

Swati Pillai

Module 2 – Practical 1: Progressive Web Applications

1a: College Information Portal as PWA

AIM

To create a simple website for a College Information Portal and convert it into a Progressive Web Application (PWA) by adding a Service Worker and Web App Manifest.

STEPS

1. Create an HTML/CSS website for the College Portal
2. Create manifest.json with app metadata
3. Create service-worker.js to cache assets
4. Register the service worker in index.html

PROGRAM CODE

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>ABC College Portal</title>
  <link rel="manifest" href="manifest.json" />
  <link rel="stylesheet" href="style.css" />
  <meta name="theme-color" content="#1F4E79" />
</head>
<body>
  <header>
    <h1>ABC College</h1>
    <nav>
      <a href="#about">About</a>
      <a href="#courses">Courses</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>
  <main>
    <section id="about"><h2>About Us</h2><p>Excellence in Education since
1990.</p></section>
    <section id="courses">
      <h2>Courses Offered</h2>
      <ul><li>BSc IT</li><li>BSc CS</li><li>BCA</li><li>MCA</li></ul>
    </section>
    <section id="contact"><h2>Contact</h2><p>Email: info@abccollege.edu</p></section>
  </main>
  <script>
    if ('serviceWorker' in navigator) {
      navigator.serviceWorker.register('/service-worker.js')
        .then(() => console.log('Service Worker registered'))
        .catch(err => console.error('SW Error:', err));
    }
  </script>
</body>
</html>
```

manifest.json

```
{
  "name": "ABC College Portal",
  "short_name": "ABC College",
  "description": "College Information Portal PWA",
```

```
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1F4E79",
  "orientation": "portrait",
  "icons": [
    { "src": "icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

service-worker.js

```
const CACHE_NAME = 'college-portal-v1';
const ASSETS = [
  '/', '/index.html', '/style.css',
  '/icons/icon-192.png', '/icons/icon-512.png'
];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => cache.addAll(ASSETS))
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response =>
      response || fetch(event.request)
    )
  );
});
```

OUTPUT

```
Browser Address Bar: localhost:5000
┌──────────────────────────────────────┐
│           ABC College                │
│  [About] [Courses] [Contact]         │
├──────────────────────────────────────┤
│ About Us                             │
│ Excellence in Education since 1990.  │
├──────────────────────────────────────┤
│ Courses Offered: BSc IT, BSc CS, BCA │
└──────────────────────────────────────┘

Browser shows: [Install App ⊕] button in address bar
DevTools → Application → Service Workers: ✅ Activated
DevTools → Application → Manifest: ✅ Valid
```

1b: PWA with Custom Offline Page

AIM

To create a PWA that displays a custom offline page when the user is not connected to the internet.

PROGRAM CODE

offline.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Offline - ABC College</title>
  <style>
    body { font-family: Arial; text-align: center; padding: 60px; background: #f0f0f0; }
    .icon { font-size: 80px; }
```

Swati Pillai

```
    h1 { color: #1F4E79; }
    p { color: #555; }
    button { background:#1F4E79; color:white; border:none;
             padding:12px 24px; cursor:pointer; border-radius:4px; }
  </style>
</head>
<body>
  <div class="icon">📡</div>
  <h1>You are Offline</h1>
  <p>No internet connection found. Please check your network settings.</p>
  <p>Some content may still be available from cache.</p>
  <button onclick="window.location.reload()">Try Again</button>
</body>
</html>
```

service-worker.js (with offline support)

```
const CACHE_NAME = 'pwa-offline-v1';
const OFFLINE_URL = '/offline.html';

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(['/', '/offline.html', '/style.css']);
    })
  );
  self.skipWaiting();
});

self.addEventListener('fetch', event => {
  if (event.request.mode === 'navigate') {
    event.respondWith(
      fetch(event.request).catch(() =>
        caches.open(CACHE_NAME).then(cache => cache.match(OFFLINE_URL))
      )
    );
  } else {
    event.respondWith(
      caches.match(event.request).then(r => r || fetch(event.request))
    );
  }
});
```

OUTPUT

```
When online:  Normal college portal page loads ✅

When offline (DevTools → Network → Offline):
┌──────────────────────────────────────┐
│              📡                      │
│         You are Offline              │
│   No internet connection found.      │
│   Please check your network settings.│
│   Some content may still be available.│
│            [Try Again]               │
└──────────────────────────────────────┘
```

Swati Pillai

Module 2 – Practical 2: Offline PWA and Student Record Management

2a: PWA with Offline Mode using Service Worker

AIM

To develop a PWA that works in offline mode using Service Worker with cache-first strategy.

PROGRAM CODE

service-worker.js (Cache First Strategy)

```
const CACHE_VERSION = 'v2';
const CACHE_NAME = `pwa-cache-${CACHE_VERSION}`;

const STATIC_ASSETS = [
  '/', '/index.html', '/style.css', '/app.js',
  '/icons/icon-192.png', '/icons/icon-512.png', '/offline.html'
];

// Install: Cache all static assets
self.addEventListener('install', event => {
  console.log('[SW] Install');
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(STATIC_ASSETS))
      .then(() => self.skipWaiting())
  );
});

// Activate: Clean old caches
self.addEventListener('activate', event => {
  console.log('[SW] Activate');
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k))
      )
    ).then(() => self.clients.claim())
  );
});

// Fetch: Cache First, Network Fallback
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      if (cached) {
        console.log('[SW] Serving from cache:', event.request.url);
        return cached;
      }
      return fetch(event.request).then(response => {
        const clone = response.clone();
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, clone));
        return response;
      }).catch(() => caches.match('/offline.html'));
    })
  );
});
```

OUTPUT

```
DevTools → Application → Service Workers:
Status: ● activated and running
```

```
Scope:  http://localhost:5000/

DevTools → Application → Cache Storage:
pwa-cache-v2 → 7 items cached

Console log when offline:
[SW] Serving from cache: http://localhost:5000/
[SW] Serving from cache: http://localhost:5000/style.css
[SW] Serving from cache: http://localhost:5000/app.js
```

2b: Student Record Management PWA

AIM

To create a Student Record Management Progressive Web Application with offline support and installability.

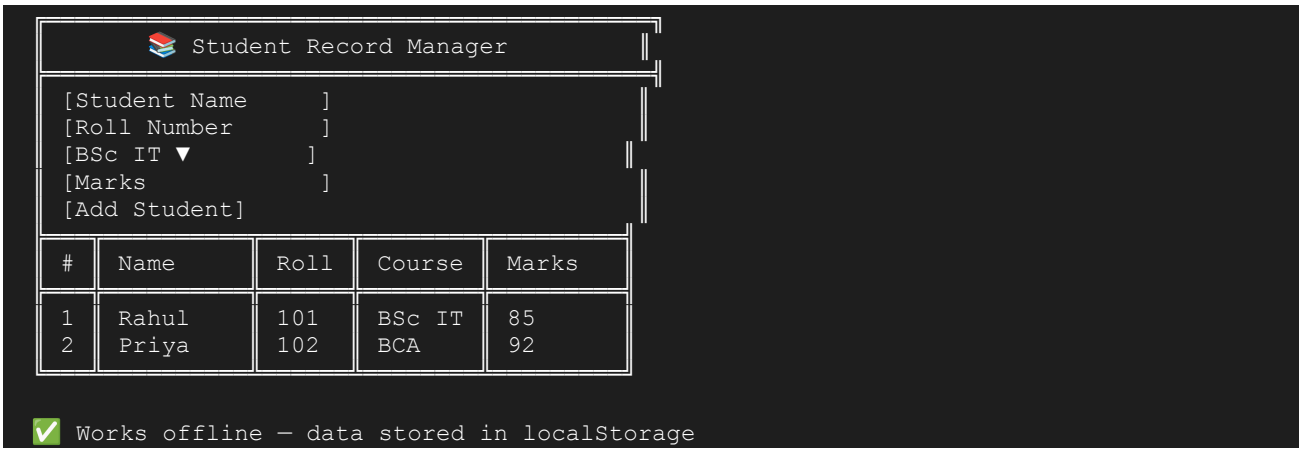
PROGRAM CODE

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Records PWA</title>
  <link rel="manifest" href="manifest.json" />
  <meta name="theme-color" content="#2E75B6" />
  <style>
    body { font-family: Arial; max-width: 600px; margin: auto; padding: 20px; }
    input, select { width: 100%; padding: 8px; margin: 5px 0; box-sizing: border-box; }
    button { background: #2E75B6; color: white; padding: 10px 20px; border: none; cursor:
pointer; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
    th { background: #2E75B6; color: white; }
  </style>
</head>
<body>
  <h2>📚 Student Record Manager</h2>
  <div>
    <input type="text" id="name" placeholder="Student Name" />
    <input type="text" id="roll" placeholder="Roll Number" />
    <select id="course">
      <option value="BSc IT">BSc IT</option>
      <option value="BSc CS">BSc CS</option>
      <option value="BCA">BCA</option>
    </select>
    <input type="number" id="marks" placeholder="Marks (out of 100)" />
    <button onclick="addStudent()">Add Student</button>
  </div>
  <table>

<thead><tr><th>#</th><th>Name</th><th>Roll</th><th>Course</th><th>Marks</th></tr></thead>
    <tbody id="studentTable"></tbody>
  </table>
  <script>
    let students = JSON.parse(localStorage.getItem('students')) || [];
    function render() {
      const tbody = document.getElementById('studentTable');
      tbody.innerHTML = students.map((s, i) =>
        `<tr><td>${i+1}</td><td>${s.name}</td><td>${s.roll}</td>
            <td>${s.course}</td><td>${s.marks}</td></tr>`
      ).join('');
    }
    function addStudent() {
```

```
      students.push({
        name: document.getElementById('name').value,
        roll: document.getElementById('roll').value,
        course: document.getElementById('course').value,
        marks: document.getElementById('marks').value
      });
      localStorage.setItem('students', JSON.stringify(students));
      render();
    }
    render();
    if ('serviceWorker' in navigator)
      navigator.serviceWorker.register('/sw.js');
  </script>
</body>
</html>
```

OUTPUT



Swati Pillai

Module 2 – Practical 3: HTML Start Page and Web API

3a: HTML Page with Link to manifest.json

AIM

To create an HTML page as the starting point of a PWA application and link it to a manifest.json file for app configuration.

PROGRAM CODE

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="My Progressive Web App" />
  <meta name="theme-color" content="#1F4E79" />

  <!-- Link to manifest.json -->
  <link rel="manifest" href="manifest.json" />

  <!-- Icon for iOS -->
  <link rel="apple-touch-icon" href="icons/icon-192.png" />

  <title>My PWA</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div id="app">
    <h1>Welcome to My PWA</h1>
    <p>This app works online and offline!</p>
    <p id="status">Checking connection...</p>
  </div>
  <script>
    // Check online status
    function updateStatus() {
      document.getElementById('status').textContent =
        navigator.onLine ? '🟢 Online' : '🔴 Offline';
    }
    window.addEventListener('online', updateStatus);
    window.addEventListener('offline', updateStatus);
    updateStatus();

    // Register Service Worker
    if ('serviceWorker' in navigator) {
      window.addEventListener('load', () => {
        navigator.serviceWorker.register('/sw.js').then(reg => {
          console.log('SW registered. Scope:', reg.scope);
        });
      });
    }
  </script>
</body>
</html>
```

manifest.json

```
{
  "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "description": "A sample PWA application",
```

```
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1F4E79",
  "lang": "en",
  "orientation": "any",
  "scope": "/",
  "icons": [
    {
      "src": "icons/icon-72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

OUTPUT

```
Browser shows the page with install prompt (⊕)
DevTools > Application > Manifest:
  App name: My Progressive Web App
  Start URL: /index.html
  Display: standalone
  Icons: 3 found ✅
  Manifest valid ✅

Page displays: 🟢 Online
(When network disabled): 🔴 Offline
```

3b: Simple Web API returning Student data in JSON

AIM

To create a simple ASP.NET Core Web API that returns Student data in JSON format.

STEPS

1. Open Visual Studio → File → New → Project → ASP.NET Core Web API
2. Name: StudentAPI → Create
3. Create Student model and Students controller

PROGRAM CODE

Models/Student.cs

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Course { get; set; }
```

```
        public int Age { get; set; }
        public double Percentage { get; set; }
    }
}
```

Controllers/StudentsController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

namespace StudentAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentsController : ControllerBase
    {
        private static List<Student> students = new List<Student>
        {
            new Student { Id=1, Name="Alice", Course="BSc IT", Age=20, Percentage=85.5 },
            new Student { Id=2, Name="Bob",   Course="BCA",    Age=21, Percentage=78.0 },
            new Student { Id=3, Name="Carol", Course="BSc CS", Age=19, Percentage=91.2 }
        };

        // GET: api/students
        [HttpGet]
        public ActionResult<IEnumerable<Student>> GetAll()
            => Ok(students);

        // GET: api/students/1
        [HttpGet("{id}")]
        public ActionResult<Student> GetById(int id)
        {
            var s = students.FirstOrDefault(x => x.Id == id);
            if (s == null) return NotFound(new { message = "Student not found" });
            return Ok(s);
        }
    }
}
```

OUTPUT

GET http://localhost:5000/api/students

```
[
  { "id": 1, "name": "Alice", "course": "BSc IT",
    "age": 20, "percentage": 85.5 },
  { "id": 2, "name": "Bob",   "course": "BCA",
    "age": 21, "percentage": 78.0 },
  { "id": 3, "name": "Carol", "course": "BSc CS",
    "age": 19, "percentage": 91.2 }
]

GET http://localhost:5000/api/students/1
{ "id": 1, "name": "Alice", "course": "BSc IT",
  "age": 20, "percentage": 85.5 }

GET http://localhost:5000/api/students/99
{ "message": "Student not found" }  (404 Not Found)
```

Swati Pillai

Module 2 – Practical 4: CRUD Web API and Factorial API

4a: CRUD Operations using ASP.NET Core Web API and EF Core

AIM

To develop CRUD (Create, Read, Update, Delete) operations using ASP.NET Core Web API and Entity Framework Core with SQL Server.

PROGRAM CODE

Controllers/EmployeesController.cs (Full CRUD API)

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebAPI.Data;
using WebAPI.Models;

namespace WebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeesController : ControllerBase
    {
        private readonly AppDbContext _ctx;
        public EmployeesController(AppDbContext ctx) { _ctx = ctx; }

        // GET api/employees
        [HttpGet]
        public async Task<IActionResult> GetAll()
            => Ok(await _ctx.Employees.ToListAsync());

        // GET api/employees/1
        [HttpGet("{id}")]
        public async Task<IActionResult> GetById(int id)
        {
            var emp = await _ctx.Employees.FindAsync(id);
            return emp == null ? NotFound() : Ok(emp);
        }

        // POST api/employees
        [HttpPost]
        public async Task<IActionResult> Create([FromBody] Employee emp)
        {
            _ctx.Employees.Add(emp);
            await _ctx.SaveChangesAsync();
            return CreatedAtAction(nameof(GetById), new { id = emp.EmployeeId }, emp);
        }

        // PUT api/employees/1
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, [FromBody] Employee emp)
        {
            if (id != emp.EmployeeId) return BadRequest();
            _ctx.Entry(emp).State = EntityState.Modified;
            await _ctx.SaveChangesAsync();
            return NoContent();
        }

        // DELETE api/employees/1
        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
```

```
        {
            var emp = await _ctx.Employees.FindAsync(id);
            if (emp == null) return NotFound();
            _ctx.Employees.Remove(emp);
            await _ctx.SaveChangesAsync();
            return NoContent();
        }
    }
}
```

OUTPUT (Tested via Swagger/Postman)

```
POST   /api/employees   → 201 Created
Body: {"name":"Rahul","department":"IT","salary":45000}

GET    /api/employees   → 200 OK
[{"employeeId":1,"name":"Rahul","department":"IT","salary":45000}]

GET    /api/employees/1 → 200 OK
{"employeeId":1,"name":"Rahul","department":"IT","salary":45000}

PUT    /api/employees/1 → 204 No Content
Body: {"employeeId":1,"name":"Rahul","department":"HR","salary":50000}

DELETE /api/employees/1 → 204 No Content
GET    /api/employees/1 → 404 Not Found
```

4b: Factorial API Endpoint tested with Postman

AIM

To create an ASP.NET Core Web API endpoint that calculates the factorial of a given number and test it using Postman.

PROGRAM CODE

Controllers/MathController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class MathController : ControllerBase
    {
        // GET api/math/factorial/5
        [HttpGet("factorial/{n}")]
        public IActionResult GetFactorial(int n)
        {
            if (n < 0)
                return BadRequest(new { error = "Number must be non-negative" });

            if (n > 20)
                return BadRequest(new { error = "Number too large (max 20)" });

            long result = CalculateFactorial(n);
            return Ok(new
            {
                number = n,
                factorial = result,
                expression = $"{n}! = {result}"
            });
        }
```

Swati Pillai

```
        private long CalculateFactorial(int n)
        {
            if (n == 0 || n == 1) return 1;
            long result = 1;
            for (int i = 2; i <= n; i++)
                result *= i;
            return result;
        }

        // POST api/math/factorial
        [HttpPost("factorial")]
        public IActionResult PostFactorial([FromBody] FactorialRequest req)
        {
            if (req.Number < 0)
                return BadRequest(new { error = "Number must be non-negative" });
            return Ok(new { factorial = CalculateFactorial(req.Number) });
        }
    }

    public class FactorialRequest
    {
        public int Number { get; set; }
    }
}
```

OUTPUT (Postman Testing)

```
Postman - Test 1: GET /api/math/factorial/5
Status: 200 OK
Response:
{
  "number": 5,
  "factorial": 120,
  "expression": "5! = 120"
}

Postman - Test 2: GET /api/math/factorial/10
Status: 200 OK
Response:
{
  "number": 10,
  "factorial": 3628800,
  "expression": "10! = 3628800"
}

Postman - Test 3: GET /api/math/factorial/-1
Status: 400 Bad Request
{
  "error": "Number must be non-negative"
}
```

Swati Pillai

Module 2 – Practical 5: PWA Manifest, Push Notifications, and E-Commerce PWA

5a: PWA with Web App Manifest and Push Notifications

AIM

To create a PWA with a complete Web App Manifest file (app name, icons, start URL, display mode) and implement Push Notifications.

PROGRAM CODE

manifest.json (Complete)

```
{
  "name": "MyApp - Full Featured PWA",
  "short_name": "MyApp",
  "description": "A fully featured Progressive Web App",
  "start_url": "/index.html?source=pwa",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait-primary",
  "background_color": "#1F4E79",
  "theme_color": "#2E75B6",
  "lang": "en-US",
  "categories": ["education", "utilities"],
  "icons": [
    { "src": "/icons/icon-72x72.png",   "sizes": "72x72",   "type": "image/png" },
    { "src": "/icons/icon-96x96.png",   "sizes": "96x96",   "type": "image/png" },
    { "src": "/icons/icon-128x128.png", "sizes": "128x128", "type": "image/png" },
    { "src": "/icons/icon-192x192.png", "sizes": "192x192", "type": "image/png",
      "purpose": "any maskable" },
    { "src": "/icons/icon-512x512.png", "sizes": "512x512", "type": "image/png",
      "purpose": "any maskable" }
  ],
  "shortcuts": [
    {
      "name": "View Students",
      "url": "/students",
      "icons": [{ "src": "/icons/students.png", "sizes": "96x96" }]
    }
  ]
}
```

push-notification.js

```
// Request notification permission
async function requestPermission() {
  const permission = await Notification.requestPermission();
  if (permission === 'granted') {
    console.log('Notification permission granted');
    subscribeUser();
  } else {
    console.log('Notification permission denied');
  }
}

// Subscribe to push notifications
async function subscribeUser() {
  const registration = await navigator.serviceWorker.ready;
  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: urlBase64ToUint8Array('YOUR_VAPID_PUBLIC_KEY')
  });
```

```
  console.log('Push subscription:', JSON.stringify(subscription));
  // Send subscription to server
  await fetch('/api/subscribe', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(subscription)
  });
}

// Show local notification demo
function showNotification() {
  if (Notification.permission === 'granted') {
    new Notification('MyApp Notification', {
      body: 'You have a new message!',
      icon: '/icons/icon-192x192.png',
      badge: '/icons/badge.png',
      tag: 'demo-notification'
    });
  }
}

// Service Worker: push event handler
// (in service-worker.js)
self.addEventListener('push', event => {
  const data = event.data ? event.data.json() : {};
  const options = {
    body: data.body || 'New notification!',
    icon: '/icons/icon-192x192.png',
    badge: '/icons/badge.png',
    vibrate: [200, 100, 200]
  };
  event.waitUntil(
    self.registration.showNotification(data.title || 'MyApp', options)
  );
});
```

OUTPUT

```
Browser permission popup:

┌──────────────────────────────────────┐
│ localhost wants to:                  │
│ Show notifications                   │
│          [Block]  [Allow]            │
└──────────────────────────────────────┘

After clicking Allow:

┌──────────────────────────────────────┐
│ 🔔 MyApp Notification        [X]     │
│ You have a new message!              │
└──────────────────────────────────────┘

DevTools → Application → Manifest: All valid ✅
DevTools → Application → Push Messaging: Subscribed ✅
```

5b: E-Commerce PWA with Product Listing and Offline Cart

AIM

To develop an E-Commerce Progressive Web Application with product listing and offline cart functionality using Service Worker and localStorage.

PROGRAM CODE

index.html (E-Commerce PWA)

Swati Pillai

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>ShopPWA - E-Commerce</title>
  <link rel="manifest" href="manifest.json" />
  <meta name="theme-color" content="#E74C3C" />
  <style>
    * { box-sizing: border-box; }
    body { font-family: Arial; margin: 0; background: #f5f5f5; }
    header { background: #E74C3C; color: white; padding: 15px 20px; display:flex;
             justify-content:space-between; align-items:center; }
    .products { display:grid; grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
                gap:20px; padding:20px; }
    .card { background:white; border-radius:8px; padding:15px;
            box-shadow:0 2px 8px rgba(0,0,0,0.1); text-align:center; }
    .card img { width:100%; height:150px; object-fit:cover; border-radius:4px; }
    .price { color:#E74C3C; font-size:1.2em; font-weight:bold; margin:8px 0; }
    .btn { background:#E74C3C; color:white; border:none; padding:8px 16px;
           border-radius:4px; cursor:pointer; width:100%; }
    .btn:hover { background:#C0392B; }
    .cart-panel { position:fixed; right:0; top:60px; background:white;
                  width:300px; height:calc(100vh-60px);
                  box-shadow:-2px 0 8px rgba(0,0,0,0.1); padding:20px;
                  overflow-y:auto; display:none; }
    .cart-panel.open { display:block; }
    #cartBadge { background:white; color:#E74C3C; border-radius:50%;
                 padding:2px 6px; font-size:12px; font-weight:bold; }
    #offlineBar { background:#FFA500; text-align:center; padding:8px;
                  display:none; }
  </style>
</head>
<body>
  <div id="offlineBar">⚠️ You are offline. Cart saved locally.</div>
  <header>
    <h2>🛍️ ShopPWA</h2>
    <button onclick="toggleCart()"
style="background:transparent;border:none;color:white;font-size:20px;cursor:pointer;">
      🛒 Cart <span id="cartBadge">0</span>
    </button>
  </header>
  <div class="products" id="productList"></div>
  <div class="cart-panel" id="cartPanel">
    <h3>Your Cart</h3>
    <div id="cartItems"></div>
    <hr />
    <p>Total: <strong id="cartTotal">₹0</strong></p>
    <button class="btn" onclick="checkout()">Checkout</button>
    <button onclick="toggleCart()" style="margin-top:10px;width:100%">Close</button>
  </div>
  <script>
    const products = [
      { id:1, name:'Laptop',    price:55000, emoji:'💻' },
      { id:2, name:'Headphones',price:2500,  emoji:'🎧' },
      { id:3, name:'Smartphone',price:18000, emoji:'📱' },
      { id:4, name:'Tablet',    price:22000, emoji:'📟' },
      { id:5, name:'Keyboard',  price:1500,  emoji:'⌨️' },
      { id:6, name:'Mouse',     price:800,   emoji:'🖱️' }
    ];

    let cart = JSON.parse(localStorage.getItem('pwa-cart') || '[]');

    function renderProducts() {
      document.getElementById('productList').innerHTML = products.map(p => `
        <div class='card'>
```

Swati Pillai

```
          <div style='font-size:80px'>${p.emoji}</div>
          <h3>${p.name}</h3>
          <div class='price'>₹${p.price.toLocaleString()}</div>
          <button class='btn' onclick='addToCart(${p.id})'>Add to Cart</button>
        </div>`).join('');
    }

    function addToCart(productId) {
      const product = products.find(p => p.id === productId);
      const existing = cart.find(item => item.id === productId);
      if (existing) existing.qty++;
      else cart.push({ ...product, qty: 1 });
      saveCart();
      renderCart();
      alert(`${product.name} added to cart!`);
    }

    function saveCart() {
      localStorage.setItem('pwa-cart', JSON.stringify(cart));
    }

    function renderCart() {
      const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0);
      document.getElementById('cartBadge').textContent = cart.reduce((s,i)=>s+i.qty,0);
      document.getElementById('cartTotal').textContent = `₹${total.toLocaleString()}`;
      document.getElementById('cartItems').innerHTML = cart.map(i => `
        <div style='display:flex;justify-content:space-between;margin:8px 0'>
          <span>${i.emoji} ${i.name} x${i.qty}</span>
          <span>₹${(i.price*i.qty).toLocaleString()}</span>
        </div>`).join('') || '<p>Cart is empty</p>';
    }

    function toggleCart() {
      document.getElementById('cartPanel').classList.toggle('open');
    }

    function checkout() {
      if (!navigator.onLine) {
        alert('You are offline. Order will be placed when you reconnect.');
      } else {
        alert('Order placed successfully! Thank you!');
        cart = [];
        saveCart();
        renderCart();
      }
    }

    window.addEventListener('offline', () => {
      document.getElementById('offlineBar').style.display = 'block';
    });
    window.addEventListener('online', () => {
      document.getElementById('offlineBar').style.display = 'none';
    });

    renderProducts();
    renderCart();

    if ('serviceWorker' in navigator)
      navigator.serviceWorker.register('/sw.js');
  </script>
</body>
</html>
```

OUTPUT

Swati Pillai